VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course:Open Source Software|UnitCoverage:1–5
Max Marks: 100

Student Name	Registration Number
Rudradeep Chakraborty	24BCE11151
Slot	**DateofSubmission**
D11	28-03-26

1. Introduction

Open source programs are one of the most important items today for computers. Web servers, for instance, are an important component of the internet, and many of those programs, as well as many other types of important software like programming tools, are completely free to use and modify.

This project reviews how Git works as a version control system and examines both the functionality of Git and its overall impact on society along with the philosophy behind using Git.

Additionally, the project will include exploring the use of open source software by utilizing Linux shell scripts.

2.PART A — ORIGIN AND PHILOSOPHY (Git)

A1. The Problem Git Was Created to Solve

A project is what Git was designed to be beneficial for; developers working together to make one application usually experience difficulties with their project. Before Git, developers who worked on large-scale collaborative projects would use public or private centralized Version Control Systems (VCS's). These VCS's never worked well, were generally slow, and were not set up for global collaboration.

Issues arose when working on the Linux kernel. Initially, the Linux kernel development team used a proprietary VCS called BitKeeper. BitKeeper was good, but it ran contrary to the openness of Linux's intent, as BitKeeper was not available for free. Following a dispute over licensing, the Linux kernel team lost access to Bitkeeper because of the dispute over licensing, which left them without an appropriate VCS to work with.

This situation pushed Linus Torvalds to design a new system. His goal was not just to replace BitKeeper, but to create something fundamentally better. The key problems he wanted to solve were: **speed and distribution**

Git introduced a way of working where every developer has a complete copy of the repository. This means we do not have to rely on a server and it makes working together faster and more reliable.

The people who made Git decided to make it open to everyone. This was done on purpose so that no one can limit what developers can do with Git in the future like they did with BitKeeper. So Git was made for a reason: developers should be in charge of the tools they use not the way around. Git is, about giving developers control over Git. Developers should be able to use Git the way they want to use Git.

A2. License — What It Actually Says

Git is released under the **GNU General Public License version 2 (GPL v2)**. This license plays a crucial role in defining what users are allowed to do with the software.

The Four Freedoms

The idea of free software is based on four key freedoms:

1. Free to run the program for any purpose
2. Free to study how it works
3. Free to modify it
4. Free to distribute copies (original or modified)

Git's GPL v2 license ensures that all these freedoms are preserved. Since the source code is openly available, users can inspect and modify it as needed.

Can a Company Modify and Sell Git?

A company can modify Git and sell it, but there is an important condition:

- If they distribute the modified version, they **must also provide the source code** under the same GPL license.

This is known as **copyleft**. It ensures that freedom is preserved across all versions of the software.

Free as in Beer versus Free as in Freedom

There is a difference between these two ideas.

Free as in beer means you do not have to pay for something.

On the hand Free as in freedom means you have the power to change and share things.

Git is about Free as in freedom.

Git is free to use. What makes it really valuable is that people can look at the source code and make changes to it.

For example companies can make Git work better for their needs but if they share their version of Git with others they cannot stop those people from looking at the source code of Git.

This is because Git represents Free as, in freedom.

Feature	**GPL License**	**MIT License**
Type	Copyleft	Permissive
Source code sharing	Mandatory	Not required
Commercial use	Allowed	Allowed
Modification rules	Must remain open	Can become closed

A3. The Ethics of Open Source

Open source is not just about code — it is about values, collaboration, and ownership.

Should All Software Be Open Source?

There are strong arguments on both sides.

Arguments for open source:

- Promotes transparency and trust
- Encourages innovation through collaboration
- Reduces dependency on monopolistic companies

Arguments against open source:

- Companies may lose competitive advantage
- Not all projects have a sustainable funding model
- Security risks if poorly maintained

A4. Community and Governance

Git has a strong and decentralized community.

- Originally created by Linus Torvalds
- Now maintained by a group of core contributors
- Development happens through mailing lists and public repositories

Unlike corporate-controlled software, Git is governed by community consensus. Contributions are reviewed before being accepted, ensuring quality and stability.

Major platforms like GitHub and GitLab have further expanded Git's ecosystem, making collaboration easier.

There have been no major forks of Git itself, which reflects its stability and strong governance.

A5. Impact and Evolution

Git has had a massive impact on the software industry.

- It is the **standard version control system** used worldwide
- Powers platforms like GitHub and GitLab
- Enables distributed development at scale

Git successfully solved the problems it was designed for:

- Faster workflows
- Better collaboration
- Strong data integrity

Its influence goes beyond version control — it has shaped how modern software is built.

However, Git is not without criticism:

- Steep learning curve for beginners
- Complex command structure

Despite this, its advantages far outweigh its limitations.

2. PART B — LINUX FOOTPRINT (Git)

Installation Method

The installation of **Git** on Linux is straightforward because it is available in almost all major distributions.

Using APT (Debian/Ubuntu)

```
sudo apt update
sudo apt install git
```

Using DNF (Fedora)

```
sudo dnf install git
```

Verify Installation

```
git --version
```

This command confirms that Git is installed and displays its version.

Key Directories

After installation, Git does not behave like traditional server software (no heavy daemon), but it still occupies important system locations.

Binary Location

```
which git
```

Output:

```
/usr/bin/git
```

This is the executable file used to run Git commands.

Configuration Files

Git uses multiple configuration levels:

1. **System-wide config**

```
/etc/gitconfig
```

2. **User-level config**

```
~/.gitconfig
```

3. **Repository-level config**

```
.git/config
```

These define settings like username, email, and behavior.

Logs and Data

Git does not store logs in `/var/log` like services. Instead:

- Repository history is stored inside: .git/

This directory contains:

- commit history
- branches
- metadata

User and Group

Git does not run as a background service. It runs under the **current logged-in user**.

verify:

```
whoami
```

Example:

```
Rudradeep
```

Why This Matters (Security)

- No root dependency means safer usage
- Each user controls their own repositories
- Reduces risk of system-wide damage

Service Management

Unlike servers such as Apache, Git does **not run as a system service**.

There is no:

```
systemctl start git
```

Instead, Git works as a **command-line tool** executed on demand.

Example:

```
git                                                                    init
git                                clone                             <repo>
git commit -m "message"
```

Update Model

Git is updated through the Linux package manager.

How Open Source Helps Here:

- Updates are released by community contributors
- Security patches are publicly reviewed
- No dependency on a single vendor

Conclusion:

I think Git is really good for using in the world because it is fast and flexible and lots of people help with it. The way Git works means that developers can work by themselves without needing to use a server, which helps them get things done faster and makes everything more reliable. Also because Git is source it gets better all the time and people fix security problems quickly.

I think people should use Git when they are making software. I would like to help make Git better especially to make it easier for people to understand and use because it can be hard for new people to learn. Overall Git is a good tool that works well with the ideas of open source both from a technical point of view and, from a philosophical point of view. I think Git is a choice because it is powerful and reliable and that is why I like Git.

3.PART C — FOSS ECOSYSTEM (Git)

Dependencies

Even though **Git** feels like a standalone tool, it actually depends on several underlying components:

- **C standard libraries (libc)**
- Shell environment (Bash)
- Compression tools (zlib)
- Cryptographic hashing (SHA-1 / SHA-256)

These dependencies ensure Git is:

- Fast
- Lightweight
- Reliable

What Git Has Enabled

Git has not just solved version control — it has **created an entire ecosystem**.

Platforms built on Git:

- GitHub : largest developer platform
- GitLab : CI/CD + collaboration
- Bitbucket : enterprise Git hosting

Tools inspired by Git:

- Distributed workflows
- Pull request systems
- CI/CD pipelines

Git has influenced how teams:

- Collaborate
- Review code
- Deploy applications

Role in the Software Stack

Git is not part of the traditional **LAMP stack (Linux, Apache, MySQL, PHP)**, but it plays a **supporting role** in development:

- Developers use Git to manage code for:
  - Web apps (PHP, Node, Python)
  - Databases
  - Servers

Community and Governance

Git has a strong, decentralized community:

- Originally created by Linus Torvalds
- Maintained by global contributors
- Development happens via mailing lists

Community Features:

- Open contributions
- Peer-reviewed changes
- Transparent discussions

Governance Model:

- Merit-based (contributors earn trust over time)
- No single corporate owner

This ensures:

- Stability
- Long-term sustainability
- Independence

5.Part D

TABLE:

Dimension	Git (Open Source)	Perforce (Proprietary)
Cost	Completely free	Expensive licensing
Security	Open for audit	Closed source
Support	Community-driven	Paid support
Freedom to modify	Full freedom	Restricted
Control	Community-based	Corporate control

Critical Analysis

Open source software like Git is really great because it is very flexible and open. The code for Git is there for everyone to see so people who write code can check it for problems and make changes to suit their needs. This makes Git very useful and reliable.

On the hand some software like Perforce is not open to everyone and you have to pay for it. This kind of software has a lot of features that big companies like. It comes with good support, from the people who made it.. It can be very expensive and it does not give users a lot of freedom. Companies that use this kind of software have to rely on the people who made it to fix problems and add features.

Final Verdict:

I think Git is a tool for using in the real world. Git gives us tools to control different versions of our work and it is also open and lets people work together. I think it would be really cool to help make Git better especially to make it easier for new developers to use Git. I like Git. I want to help make Git better, for everyone.

6.Part E:

SCRIPT 1 — System Identity Report

CODE:

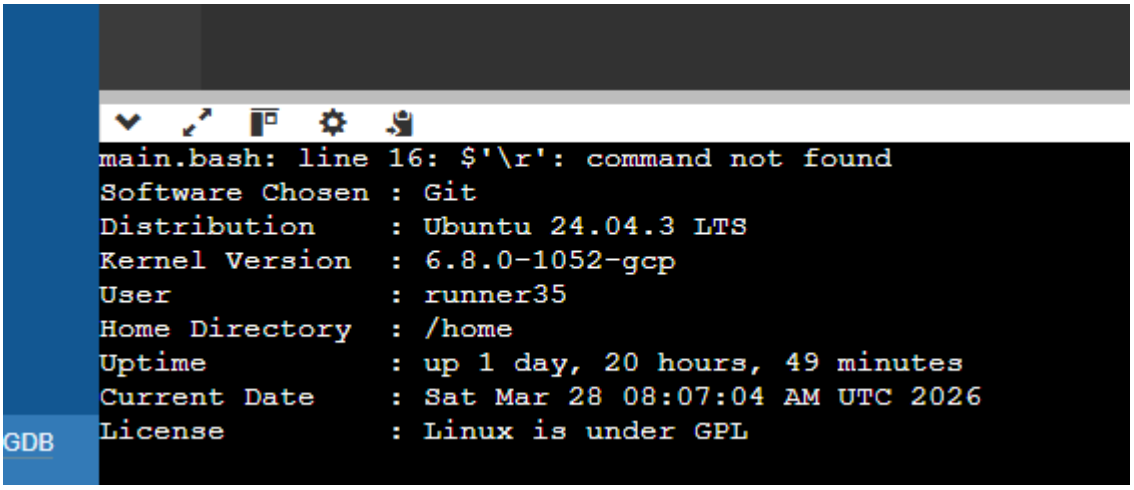
```
#!/bin/bash
# Script 1: System Identity Report
# Author: RUDRADEEP CHAKRABORTY | Course: Open Source Software
STU_NAME="RUDRADEEP CHAKRABORTY"
SOFTWARE_CHOICE="Git"
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE=$(date)

# Get distro name
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d= -f2 | tr -d '"')


#DISPLAY
echo " Open Source Audit — $STUDENT_NAME"

echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "User            : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "Uptime          : $UPTIME"
echo "Current Date    : $DATE"
echo "License         : Linux is under GPL"
```

OUTPUT:



SCRIPT 2 — FOSS Package Inspector

```
#!/bin/bash
# Script 2: FOSS Package Inspector
PACKAGE="git"

#CHECK
if dpkg -l | grep -qw $PACKAGE; then
    echo "$PACKAGE is installed."
```

```
    dpkg -s $PACKAGE | grep -E 'Version|Maintainer|Description'
else
    echo "$PACKAGE is NOT installed."
fi
#CASE AS DESCRIPTION
case $PACKAGE in
    git) echo "Git: has distributed version control" ;;
    apache2) echo "Apache: powerful." ;;
    mysql) echo "MySQL: Popular open-source database system." ;;
    vlc) echo "VLC: Free multimedia player supporting all formats." ;;
    *) echo "No description available." ;;
Esac
```

SCRIPT 3 — Disk and Permission Auditor

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
DIRS=("/main ")

echo "DA Report"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then

        # Get permissions, owner, group
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')

        # Get size
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done
# Check Git config directory
echo ""
echo "Check Git""

if [ -f "$HOME/.gitconfig" ]; then
    ls -l $HOME/.gitconfig
else
    echo "Git config file not found."
Fi
```

SCRIPT 4 — Log File Analyzer

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./script4.sh /var/log/syslog error
LOGFILE=$1
KEYWORD=${2:-"error"}
```

```
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

#LOOP FOR COUNT
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

SCRIPT 5 — Open Source Manifesto Generator

CODE:

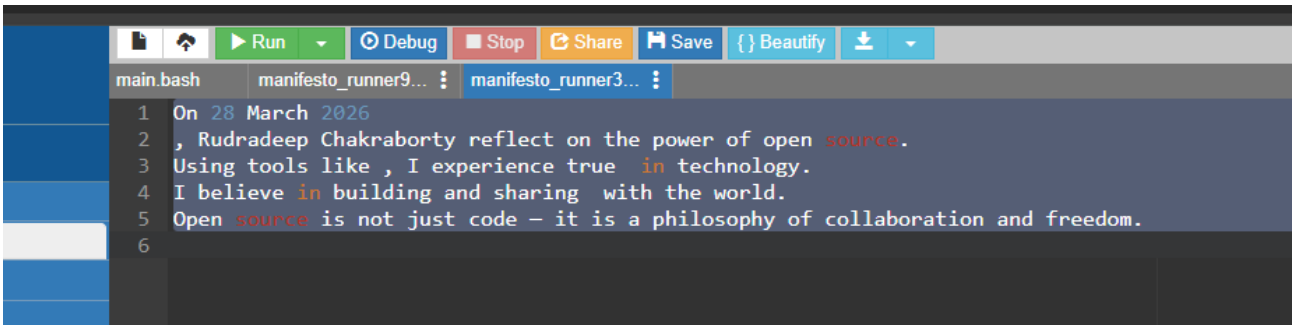
```
#USER INPUT
echo "Answer three questions to generate your manifesto."
echo ""
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

#DATE AND TIME
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

#WRITE
echo "On $DATE, Rudradeep Chakraborty reflect on the power of open source." >
$OUTPUT
echo "Using tools like $TOOL, I experience true $FREEDOM in technology." >>
$OUTPUT
echo "I believe in building and sharing $BUILD with the world." >> $OUTPUT
echo "Open source is not just code — it is a philosophy of collaboration and
freedom." >> $OUTPUT

#OUTPUT
echo ""
echo "Manifesto saved as $OUTPUT"
cat $OUTPUT
```

OUTPUT:



FINAL CONCLUSION:

This project helped me understand open-source software well both in theory and in practice.

When I studied Git I saw how open collaboration and transparency make innovation happen.

The analysis of Linux showed how open-source tools work in systems. The tasks on shell scripting helped me get better at doing things

This project shows that open-source philosophy is very important, in making software today.